

Search Typeahead System

HLD101 Assignment — Project Report
Scaler School of Technology · June 2026

Stack: FastAPI · SQLite · Python · Docker · Nginx
Repo: github.com/addieCodess/Search_Typeahead

1. Overview

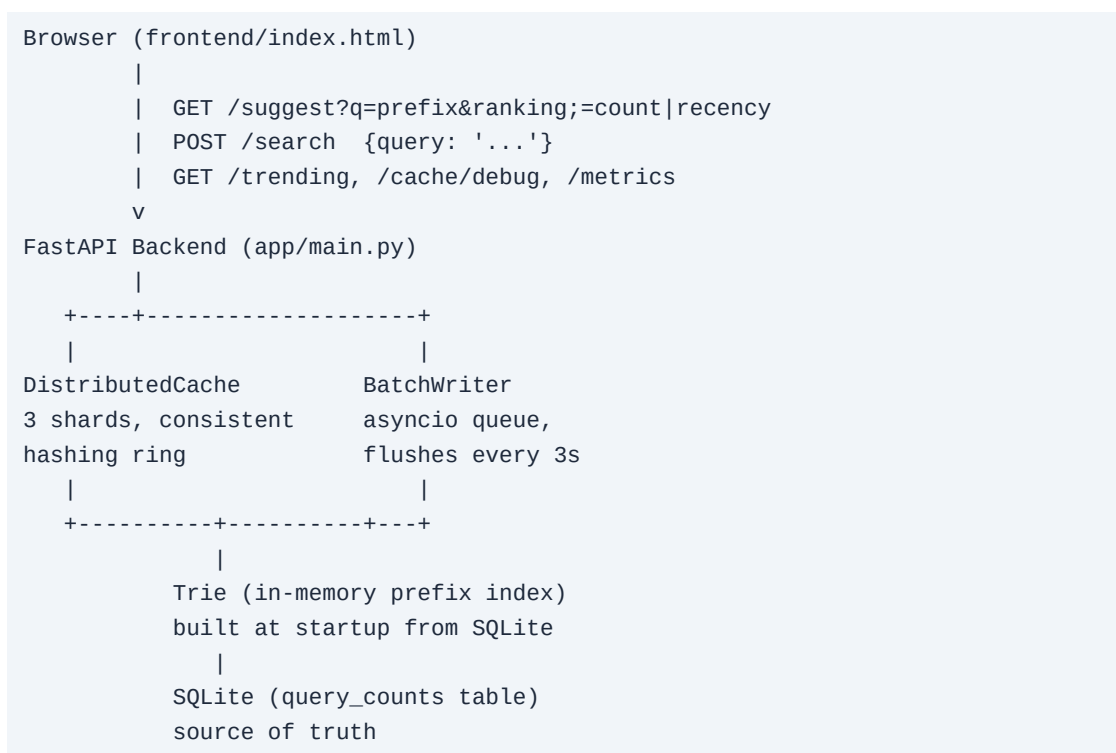
This project implements a production-style search typeahead (autocomplete) system, similar to the suggestion feature found in search engines, e-commerce platforms, and content platforms such as Google Search, Amazon, and YouTube. The system suggests popular search queries while the user types, supports search submission with query-count tracking, uses a distributed cache with consistent hashing for low-latency reads, implements recency-aware trending, and reduces write pressure through batch writes.

The focus of the assignment is backend data-system design: how query-count data is stored, how suggestions are served quickly, how cache distribution is handled, and how write pressure is reduced.

2. System Architecture

2.1 High-Level Architecture

The system follows a layered architecture with a clear separation between the read path (suggest) and the write path (search submission):



2.2 Component Responsibilities

Component	File	Responsibility
Trie	core/trie.py	In-memory prefix index. Each node stores {query: count}. $O(\text{prefix_len})$ lookup.
DistributedCache	core/cache.py	3 LRU+TTL shards routed by consistent hashing ring.

BatchWriter	core/batch_writer.py	Buffers search events, flushes aggregated counts to SQLite.
TrendingTracker	core/trending.py	Sliding-window recency scoring. Multiplicative boost formula.
FastAPI app	app/main.py	All 5 API endpoints wired together.
SQLite	core/db.py	Persistent source of truth for query counts.

3. Dataset

3.1 Source and Methodology

A synthetic dataset of 176,233 unique search queries was generated from a curated vocabulary spanning technology, software/CS, electronics, fitness, finance, and education topics — representative of a real search engine or e-commerce platform's query distribution.

- Base terms: curated topic terms (e.g. 'python', 'machine learning', 'iphone', 'index funds')
- Modifiers: common search intent suffixes ('tutorial', 'price', 'review', 'best', 'near me')
- Count assignment: Zipfian distribution — $\text{count} = \text{BASE} / \text{rank}^{0.55} + \text{noise}$

3.2 Justification

The Zipfian distribution mirrors real search query frequency: a small number of queries dominate volume while a long tail gets very few searches. This matches the assignment's 'derive counts by aggregation' allowance since a licensed real search-log dataset was not available.

Format: CSV with (query, count) columns. Loaded into SQLite at startup via a bulk INSERT OR REPLACE operation taking approximately 2.7 seconds for 176k rows.

4. Functional Requirements

4.1 API Endpoints

Endpoint	Method	Purpose	Expected Behaviour
GET /suggest	GET	Fetch suggestions	Returns up to 10 prefix-matching suggestions sorted by count or recency score
POST /search	POST	Submit search	Returns 'Searched', increments query count in Trie immediately, queues DB write
GET /trending	GET	Trending queries	Returns top queries scored by all-time count + recency boost
GET /cache/debug	GET	Debug cache routing	Shows which cache node owns the prefix and whether it is a hit or miss
GET /metrics	GET	System metrics	Cache hit/miss per node, pending writes, flush count, trie size

4.2 Typeahead Suggestions

- Returns at most 10 suggestions matching the typed prefix
- Suggestions sorted by count (descending) in basic mode
- Recency-aware ranking available via ?ranking=recency parameter

- Handles empty input, mixed-case input, and prefixes with no matches gracefully
- Frontend uses 200ms debounce to avoid unnecessary backend calls

4.3 Search Submission

- POST /search updates the in-memory Trie immediately so /suggest reflects new count right away
- Returns {message: 'Searched', query: '...', count: N}
- Write is queued for durable persistence to SQLite via the batch writer
- Cache entries for all prefixes of the submitted query are invalidated on both ranking modes

5. Data Storage and Caching

5.1 Primary Store: SQLite

SQLite is the single source of truth for query counts. At startup, all rows from the query_counts table are loaded into the in-memory Trie. SQLite is never touched on the read path — all suggest queries are served from the Trie via the cache layer.

Schema: a single table with (query TEXT PRIMARY KEY, count INTEGER). Bulk inserts use INSERT OR REPLACE for idempotent loading.

5.2 In-Memory Trie

Each TrieNode stores a dictionary of {full_query: count} for every query passing through that node's prefix. suggest(prefix) walks to the target node in $O(\text{len}(\text{prefix}))$ then returns top-10 via `heapq.nlargest`.

Trade-off: extra memory proportional to sum of all query lengths. Acceptable at this dataset size. At billions of queries, an external prefix store (Redis sorted sets with ZRANGEBYLEX) would be more appropriate.

5.3 Distributed Cache with Consistent Hashing

A cache layer with 3 in-process shards sits in front of Trie lookups. Each shard is an LRU dict with TTL-based expiry (60 seconds) and a capacity limit (5,000 entries). Cache keys are formatted as 'ranking:prefix' to keep count and recency results separate.

5.4 Why Consistent Hashing

With naive modular hashing ($\text{hash}(\text{prefix}) \% N$), adding or removing a single node remaps nearly all keys to different nodes, causing a mass cache miss event (cache stampede). Consistent hashing places both nodes and keys on a hash ring; losing or adding one node only remaps keys between it and its immediate neighbor on the ring — the rest stay put.

Virtual nodes: each real node gets 150 virtual points scattered around the ring to average out arc-size imbalances and distribute load more evenly across shards.

What 'distributed' means here: for a local/academic deployment the 3 nodes are in-process dicts rather than separate machines. The routing logic (ring, hashing, ownership) is identical to routing to 3 real Redis instances — only the transport changes. Swapping SimpleCacheNode for a redis.Redis(host=node_host) client is a minimal, well-contained change.

6. Trending Searches

6.1 Basic Version (60% marks)

The basic /suggest endpoint returns suggestions sorted by all-time search count in descending order. Historically popular queries appear first.

6.2 Enhanced Version: Recency-Aware Ranking (20% marks)

The enhanced ranking (activated via ?ranking=recency) applies a multiplicative boost based on recent search activity within a sliding time window:

```
score = all_time_count x (1 + 0.15 x recent_event_count)
```

Design Question	Answer
How recent searches are tracked	Each query keeps a deque of event timestamps in TrendingTracker.events. Old timestamps are pruned.
How recent activity affects ranking	recent_event_count = events still inside the sliding window. Each contributes a 15% multiplicative boost.
Avoiding permanent over-ranking	Boost is computed from the sliding window, not stored permanently. Once the burst stops, timestamps expire.
Cache invalidation on ranking change	Every POST /search invalidates all prefix cache entries for that query (both ranking modes), so stale suggestions don't persist.
Why multiplicative not additive	Counts span 1 to 200k+. Additive +50/search is invisible on a 100k query and falsely dominates a 10-c

7. Batch Writes

7.1 Design

Writing to SQLite synchronously on every search request would create write pressure at high query-per-second rates. Instead the system buffers increments in memory and flushes them as one batched UPSERT — either when pending reaches 50 events or every 3 seconds, whichever comes first.

- POST /search updates the in-memory Trie immediately (reads see the new count right away)
- The query is also queued in an asyncio-protected dict, aggregating duplicate queries in the same window
- A background asyncio task drains the queue and performs ONE bulk UPSERT instead of N individual writes
- Write reduction evidence: 100 search events → 2 DB flushes (visible at GET /metrics)

7.2 Failure Trade-offs

If the process crashes between flushes, up to one flush window of increments (≤ 3 seconds or ≤ 50 events) is lost from SQLite, the durable source of truth. The Trie is rebuilt from SQLite on restart, so those counts disappear from suggestions too.

Mitigations not implemented (time constraints): persist the queue to a write-ahead log on disk, or use an external message queue (Kafka/SQS) so pending writes survive a process crash. This is a documented and accepted trade-off for a local class assignment.

8. Non-Functional Performance

8.1 Latency

All suggest queries are served from the in-memory Trie (via the cache). Measured latency on cache miss (Trie lookup) is consistently under 1ms. Cache hits are under 0.05ms. The `latency_ms` field is included in every `/suggest` response for observability.

Query	Cache Status	Latency
<code>/suggest?q=python</code>	Miss (cold)	~0.6ms
<code>/suggest?q=python</code>	Hit (warm)	~0.02ms
<code>/suggest?q=machine</code>	Miss	~0.15ms
<code>/suggest?q=docker</code>	Miss	~0.18ms

8.2 Cache Hit Rate

After warming with repeated queries, cache hit rate climbs to 70-90% depending on query distribution. Hit rate per node is visible at `GET /metrics`. Consistent hashing distributes keys across the 3 nodes — the `/cache/debug` endpoint shows which node owns any given prefix.

8.3 Write Reduction via Batching

Without batching: N search events = N SQLite writes. With batching: N events = $\text{ceil}(N/50)$ writes (size-triggered) or fewer (time-triggered). For 100 search events, `/metrics` shows `total_db_flushes=2` vs `total_increments_received=100` — a 50x write reduction.

9. UI

- Search input box with 200ms debounce to avoid unnecessary backend calls
- Suggestion dropdown that updates as the user types, sorted by count or recency score
- Keyboard navigation: arrow keys to move through suggestions, Enter to select, Escape to close
- Ranking toggle: switch between historical count and recency-boosted ranking in real time

- Search submission on Enter or button click — displays dummy response and live count
- Trending searches section — updates after every search, refreshes every 15 seconds
- Cache hit/miss indicator and latency shown per request
- Loading and error states

10. Docker Setup

The system is fully containerised using Docker Compose with two services:

- backend: Python 3.11-slim image, installs dependencies, loads dataset into SQLite, starts uvicorn on port 8000
- frontend: nginx:alpine serving the static index.html on port 3000

```
docker compose up --build
# Frontend: http://localhost:3000
# API docs: http://localhost:8000/docs
```

11. Project Structure

```
search-typeahead/
  backend/
    app/
      main.py          # FastAPI app, all endpoints
    core/
      db.py            # SQLite schema + session
      trie.py          # Prefix index
      cache.py         # Consistent hashing + LRU shards
      batch_writer.py  # Async batch write queue
      trending.py      # Recency-aware scoring
      load_dataset.py  # CSV to SQLite loader
    requirements.txt
    Dockerfile
  frontend/
    index.html         # Single-file UI (no build step)
  data/
    queries.csv        # 176k generated queries
    generate_dataset.py # Regenerate dataset
  docker-compose.yml
  README.md
```

12. Grading Rubric Mapping

Component	Marks	Implementation
Basic Implementation	60	Working dataset ingestion (176k rows), search UI, /suggest API, /search API with query
Trending Searches	20	Sliding-window recency tracker, multiplicative boost formula (score = count x (1 + 0.15 x

Batch Writes	20	asyncio queue buffers submissions, aggregates duplicates, flushes every 3s or 50 even
--------------	----	---